

ECSE-4040

Digital Electronics

Lecture 9

Prof. T. Paul Chow

Bldg. CII, Room 6111

Rensselaer Polytechnic Institute

Troy, NY 12180-3590

Tel. (518)276-2910

FAX (518)276-2990

e-mail: chowt@rpi.edu

<https://lms.rpi.edu>

Lecture Outline

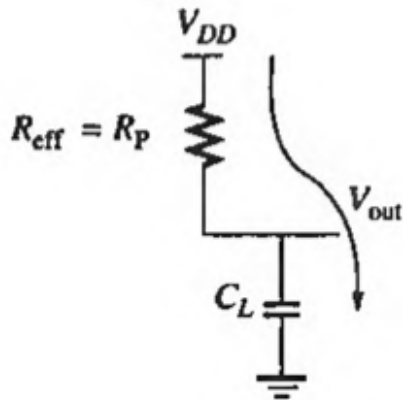
- Propagation and rise time delays
- Static MOS Logic Circuits
- Quick Review on Combinational Logic
- Flip Flops and Latches

Inverter RC delay

- Remember Physics II and Electric Circuits?
- We are just charging, discharging capacitors

$$V_C(t) = V_{C,final} \left(1 - e^{-\frac{t}{RC}}\right) \text{when charging}$$

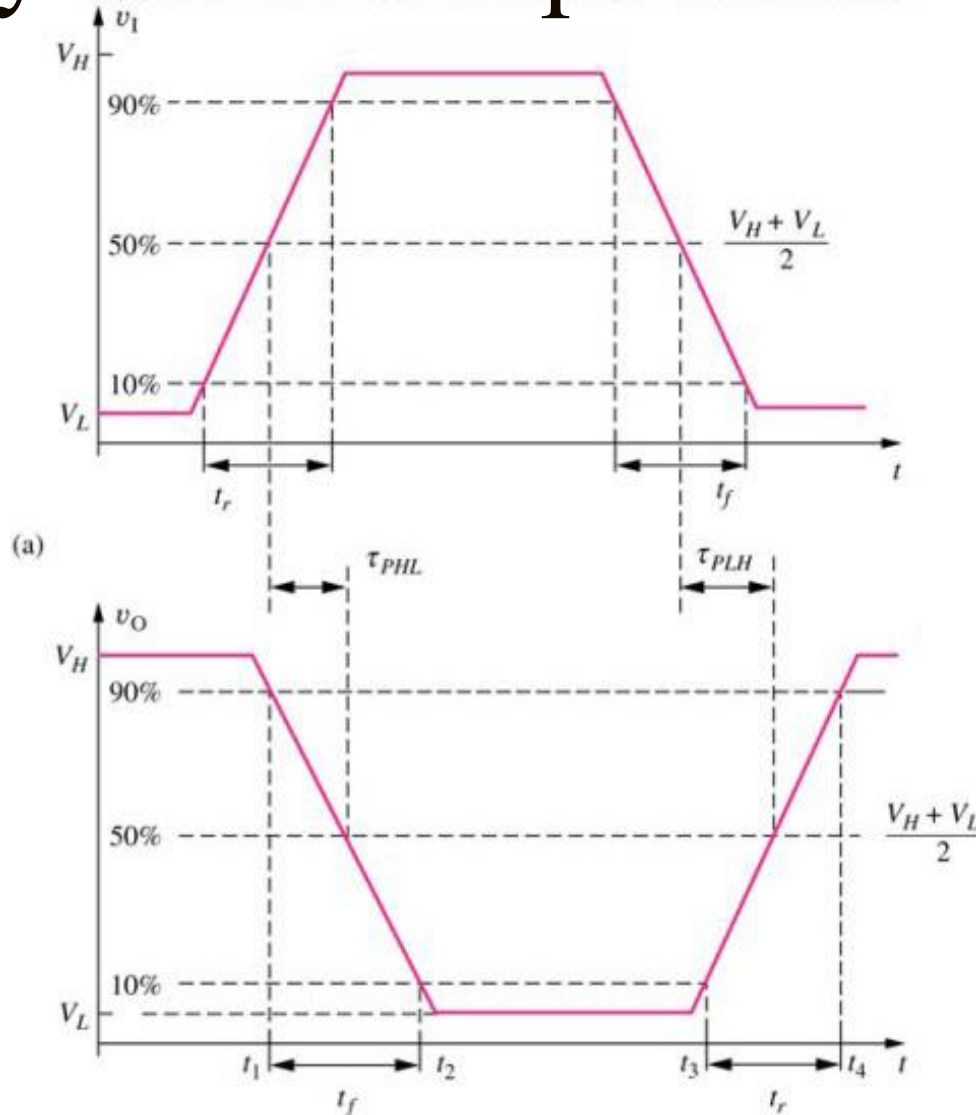
$$V_C(t) = V_{C,initial} \left(-e^{-\frac{t}{RC}}\right) \text{when discharging}$$



$$V_{out}(t) = V_{DD} \left(1 - e^{-t/R_P C_L}\right)$$

Pull-up (output capacitor charging)
through the PMOS transistor

Dynamic Response of Logic Gates



Dynamic Response of Logic Gates

- An important figure of merit to describe logic gates is their response in the time domain
- The rise and fall times, t_f and t_r , are measured at the 10% and 90% points on the transitions between the two states as shown by the following expressions:

$$V_{10\%} = V_L + 0.1\Delta V$$

$$V_{90\%} = V_L + 0.9\Delta V = V_H - 0.1\Delta V$$

Propagation Delay

- Propagation delay describes the amount of time between a change at the 50% point input to cause a change at the 50% point of the output described by the following:

$$V_{50\%} = \frac{V_H + V_L}{2}$$

- The high-to-low prop delay, τ_{PHL} , and the low-to-high prop delay, τ_{PLH} , are usually not equal, but can be described as an average value:

$$\tau_P = \frac{\tau_{PHL} + \tau_{PLH}}{2}$$

Dynamic Response of the NMOS Inverter with a Resistive Load

- The rise and fall times and propagation delays are given by the relationships:

$$t_r = 2.2RC$$

$$\tau_{PLH} = 0.69RC$$

where R and C are the resistance and capacitance seen at the output

Inverter Sizing

- The value of T_{PHL} or T_{PLH} determine the value of the resistor (average on-resistance)
- Note that in the pull-down case (NMOS on) device starts in saturation (V_{DD} across its V_{DS} and might enter triode at some point, therefore it makes sense to define an effective resistance

$$R_{eqn} = 12.5 \text{ k}\Omega/\square$$

$$R_{eqp} = 30 \text{ k}\Omega/\square$$

Unit resistance of the devices (per square)

That is for a device with $L=W$ (obtained from SPICE simulation), note the ration 2.4, does it remind of a number?

$$R_N = R_{eqn} \times \frac{L_N}{W_N}$$

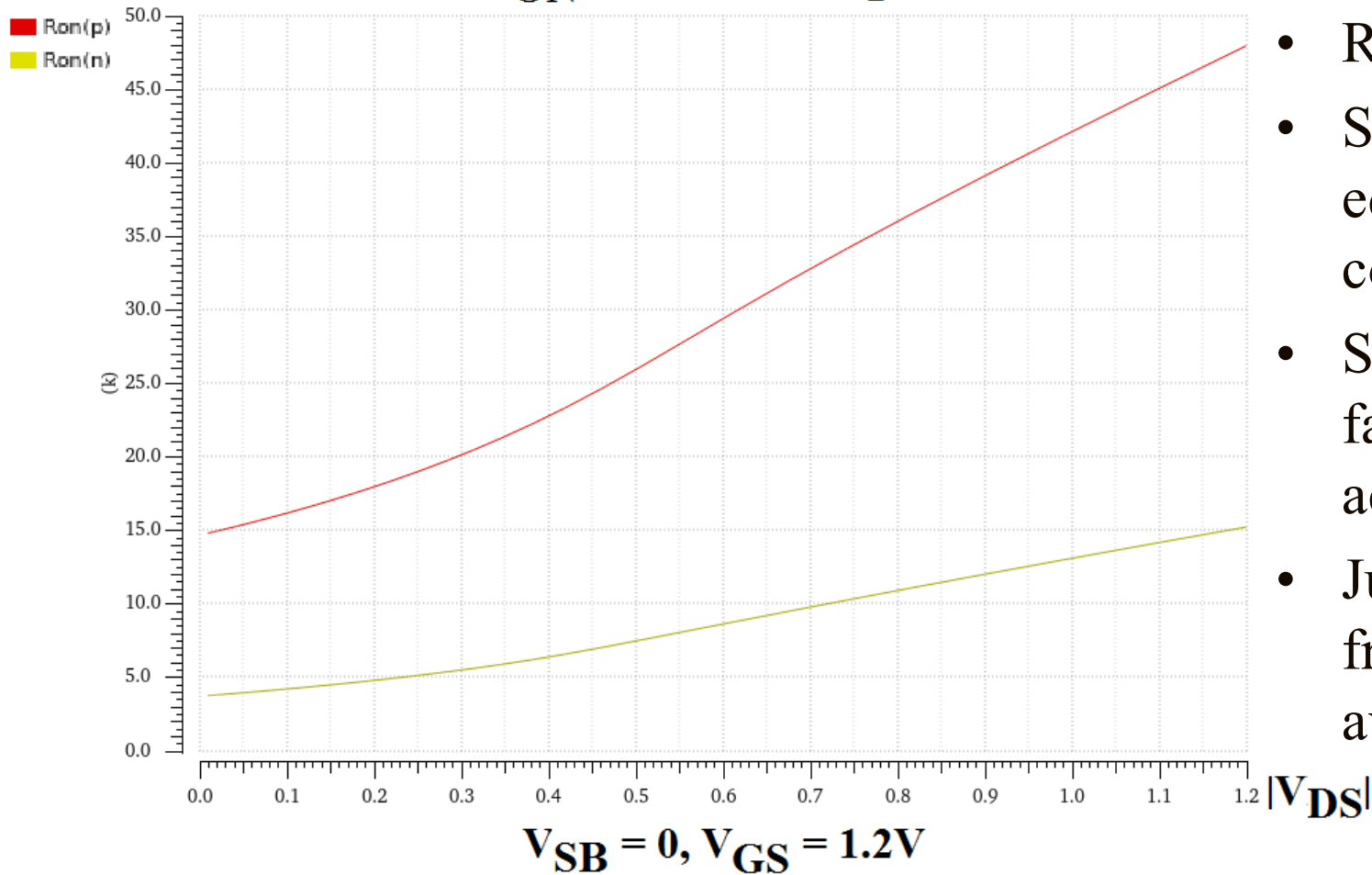
$$R_P = R_{eqp} \times \frac{L_P}{W_P}$$

Total Resistance inversely proportional to W/L (remember that in triode region we could model the transistor simply with a resistor!

Review Ex. 4.9 of HJS

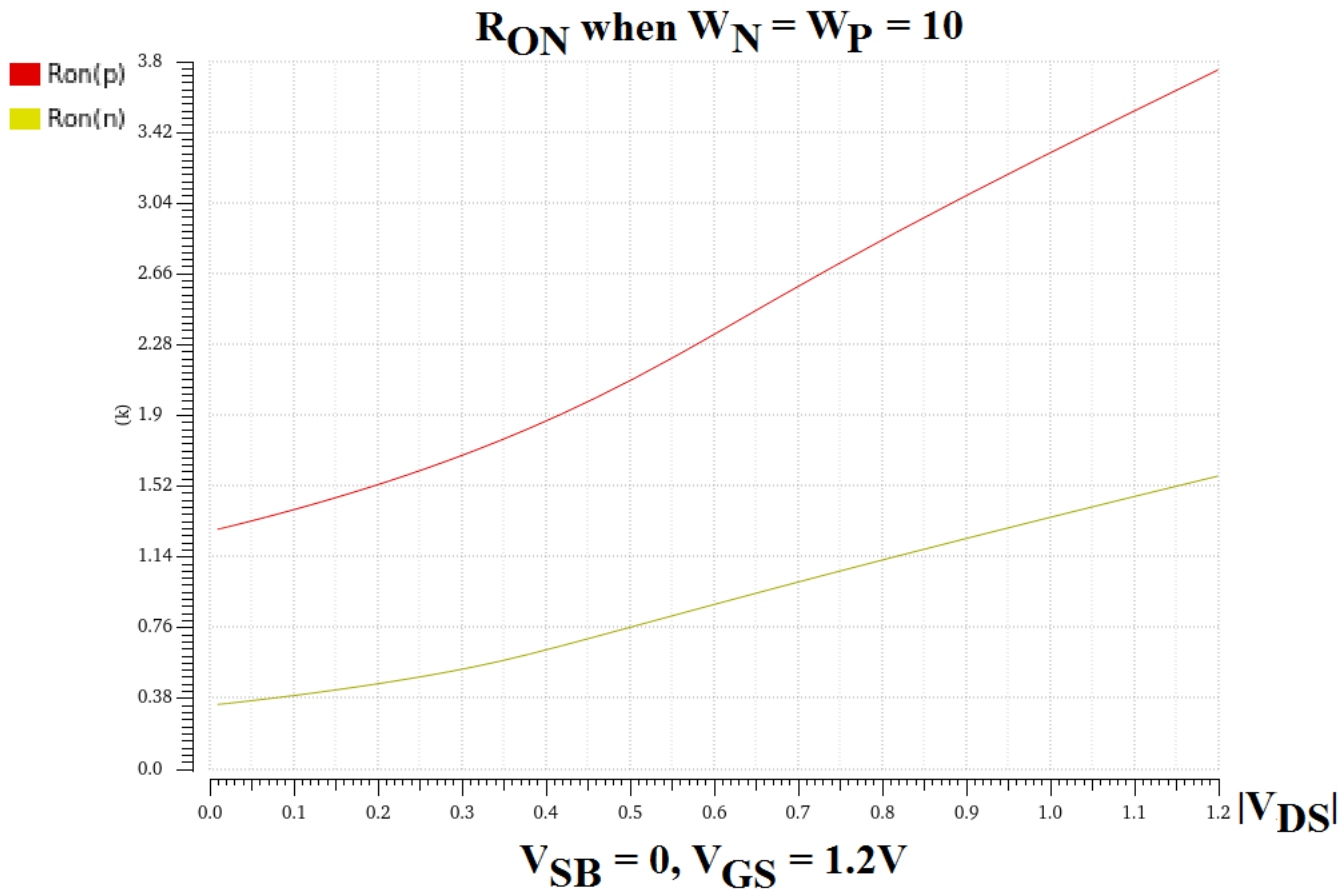
R_{ON}

R_{ON} when $W_N = W_P = 1$



- R_{ON} is V_{DS}/I_{DS}
- Solving I_{DS} equations time consuming
- Simulations faster, more accurate
- Just take R_{ON} from a table if available

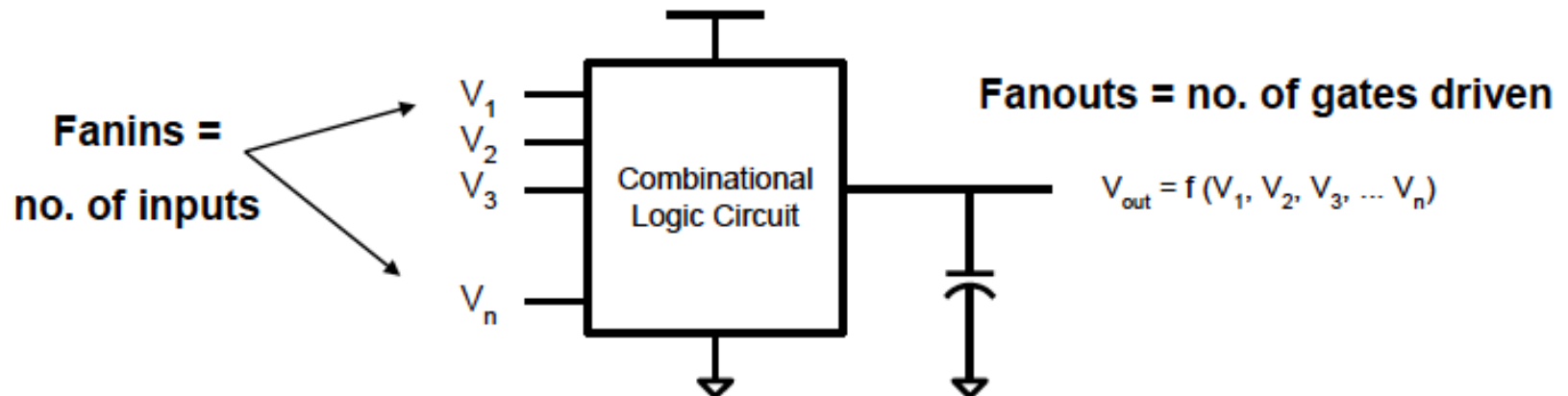
R_{ON}



- 10x width is 10x more current
- $1/10^{\text{th}} R_{ON}$

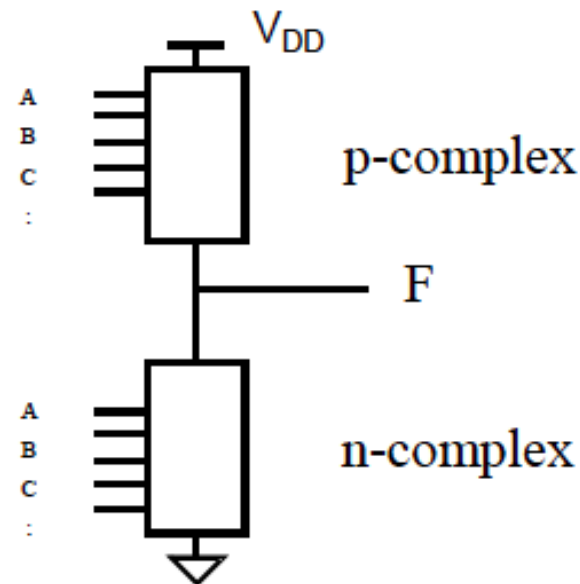
Combinational MOS Logic- Review

- Now that we understand the logic abstraction and the properties of valid logic gates, we can consider the issues of design basic building blocks of digital systems
- Typical combinational gate is a multiple input – single output system
- Performs Boolean operations on multiple input variables, drives one or more gates
- Design parameters and considerations:
 - Propagation delay
 - Static and dynamic power
 - Area
 - Noise margins (VTC)



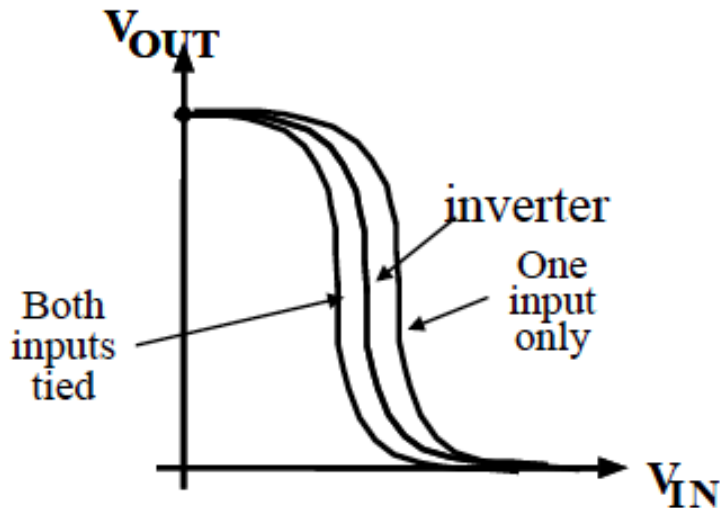
Pull-up and Pull-down Network- Review

**Basic Structure
Of all CMOS logic gates**



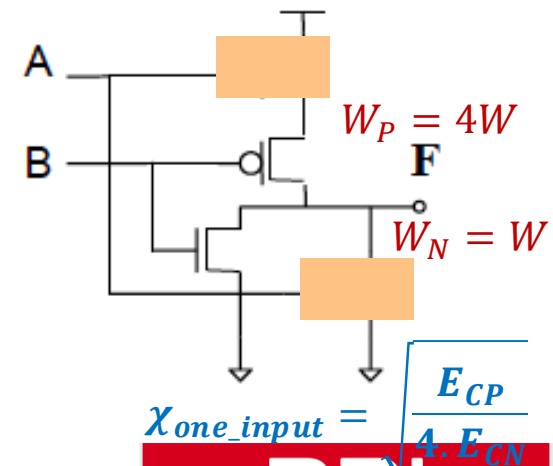
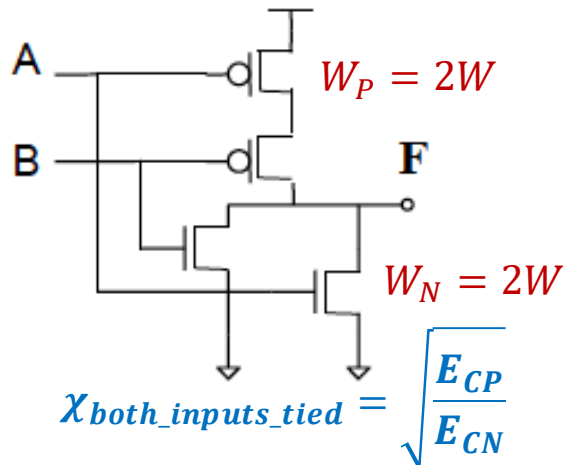
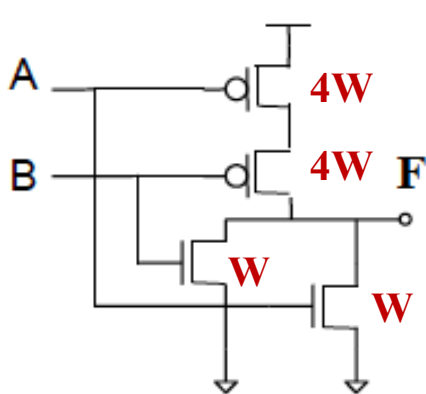
- PMOS pull-up and NMOS pull-down networks are duals of each other
- Configuration of pull-up and pull-down networks create a current connection from the output to *either* V_{DD} or GND , based on the inputs
- PMOS devices have lower drive capability and thus require wider devices to achieve the same on-resistance as its pull-down counterpart

VTC and Noise Margins-NOR gate

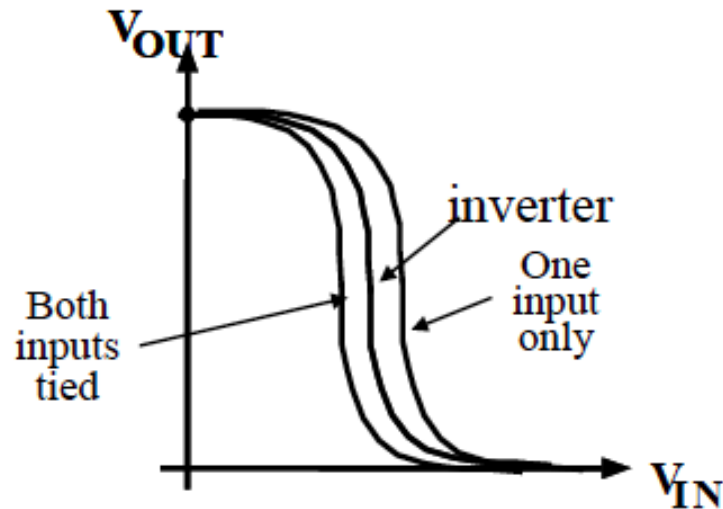


$$\chi = \sqrt{\frac{W_N/E_{CN}L_N}{W_P/E_{CP}L_P}}$$

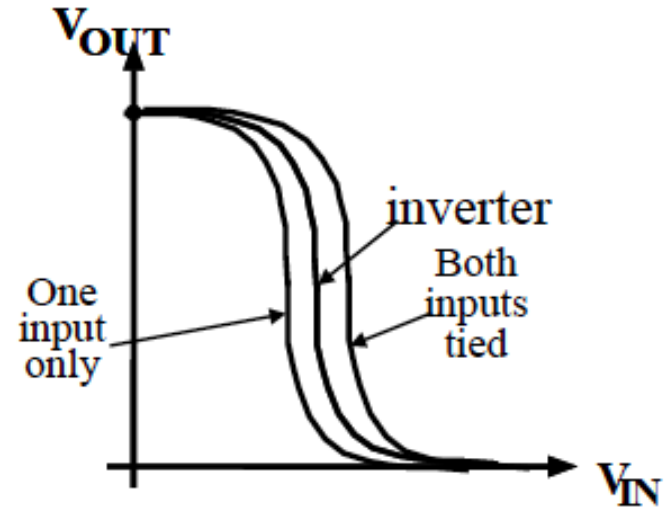
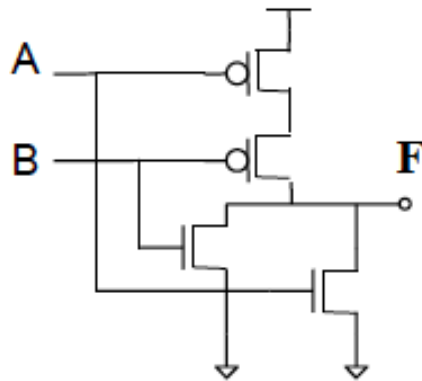
$$V_S = \frac{V_{DD} - |V_{TP}| + \chi \cdot V_{TN}}{1 + \chi}$$



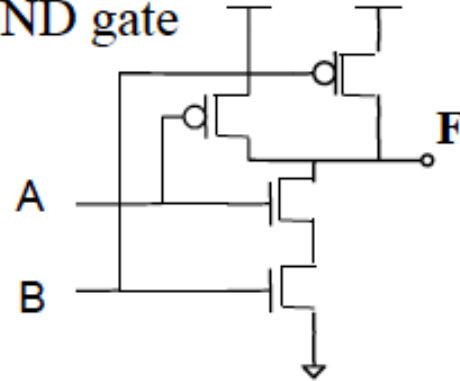
VTC and Noise Margins-Review



NOR gate

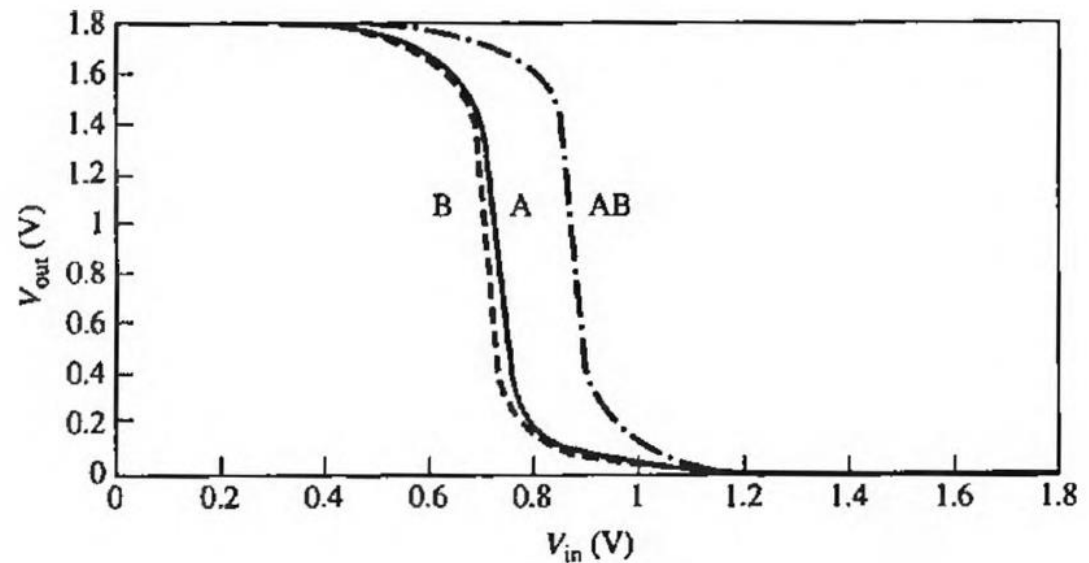
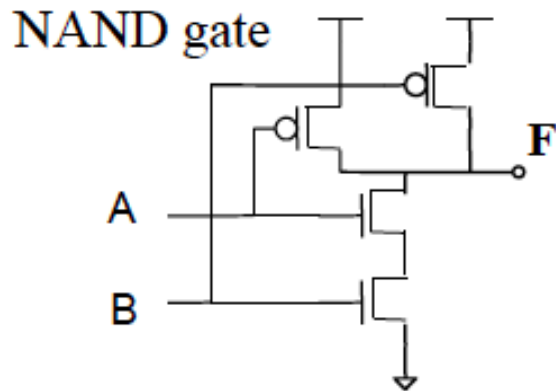
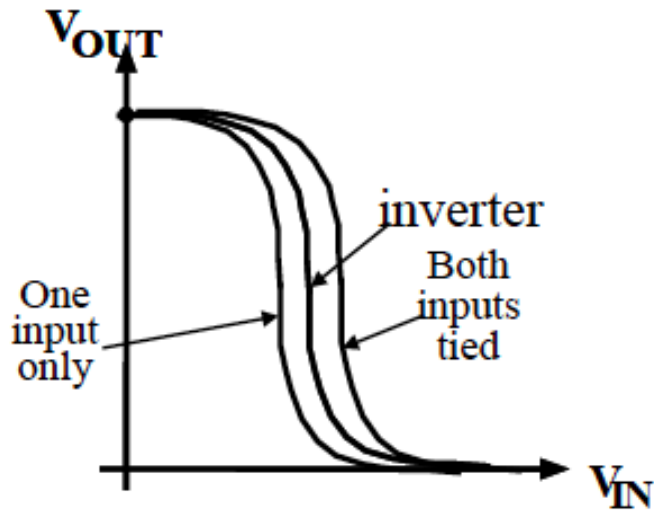


NAND gate



VTC and Noise Margins-Review

SPICE SIMULATIONS



Why do you think performance for A is different from B?

Complex Logic Circuits-Review

- Ability to easily build complex logic gates is one of the most attractive features of MOS logic circuits
- Design principle of pull-down network
 - OR operations are performed by parallel connected drivers
 - AND operations are performed by series connected drivers
 - Inversion is provided by the nature of MOS circuit operation
- Don't get too carried away...Use this knowledge wisely
 - Remember that complex functions don't have to be implemented with a single gate
 - Can break up very complicated Boolean expressions into a cascade of gate stages
- We will use DeMorgan's Law to build dual networks

Complex CMOS Gate Design-Review

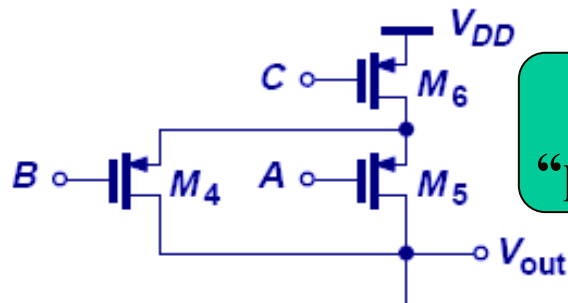
- Implement an AND-OR-INVERT (AOI) function

$$Z = \overline{(A \cdot B + C)}$$

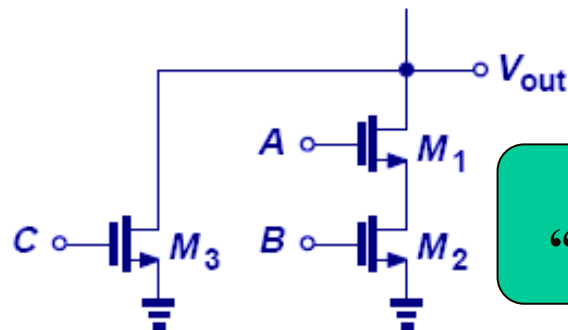
- Get the expression into forms that enable easy implementation of pull-up and pull-down networks

$$Z = (A \cdot B + C)$$

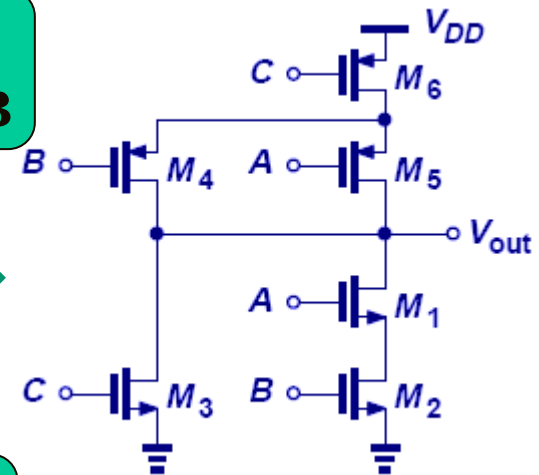
$$Z = (\overline{A} + \overline{B}) \cdot \overline{C}$$



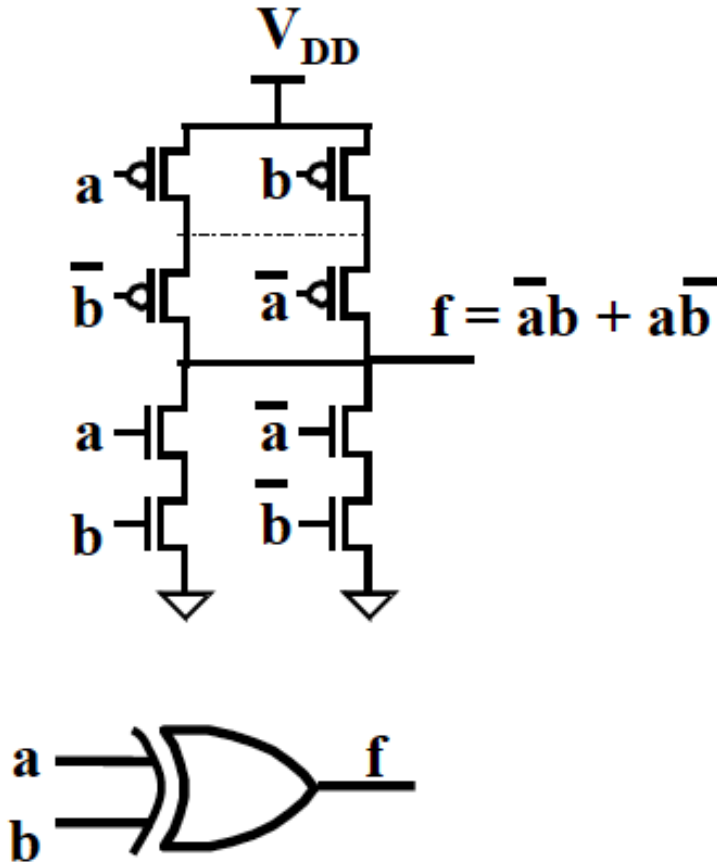
C is in “series” with the “parallel” combination of A and B



C is in “parallel” with the “series” combination of A and B



XOR Gates - Review



$$f = \overline{a} \cdot b + a \cdot \overline{b}$$

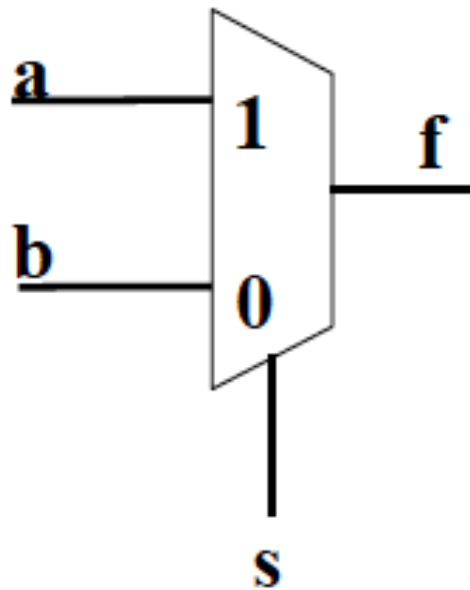
$$f = (a + \overline{b}) \cdot (\overline{a} + b)$$

$$f = \overline{a} \cdot \overline{a} + a \cdot b + \overline{a} \cdot b + \overline{b} \cdot b$$

$$f = \cancel{\overline{a} \cdot \overline{a}} + a \cdot b + \overline{a} \cdot b + \cancel{\overline{b} \cdot b}$$

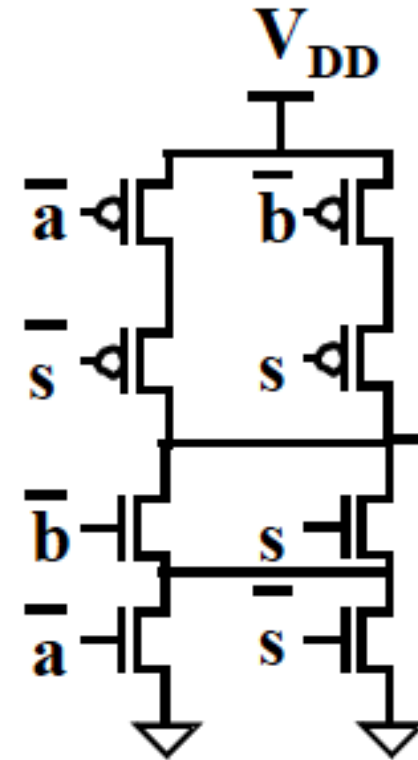
$$f = a \cdot b + \overline{a} \cdot \overline{b}$$

CMOS Multiplexer



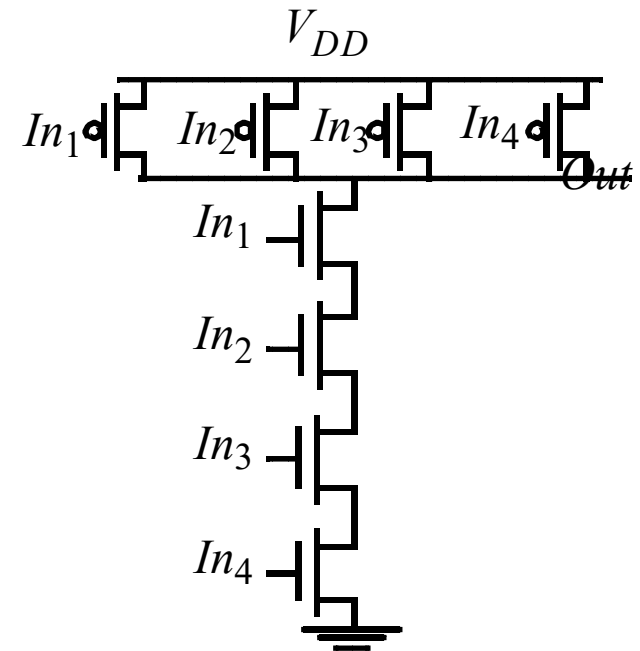
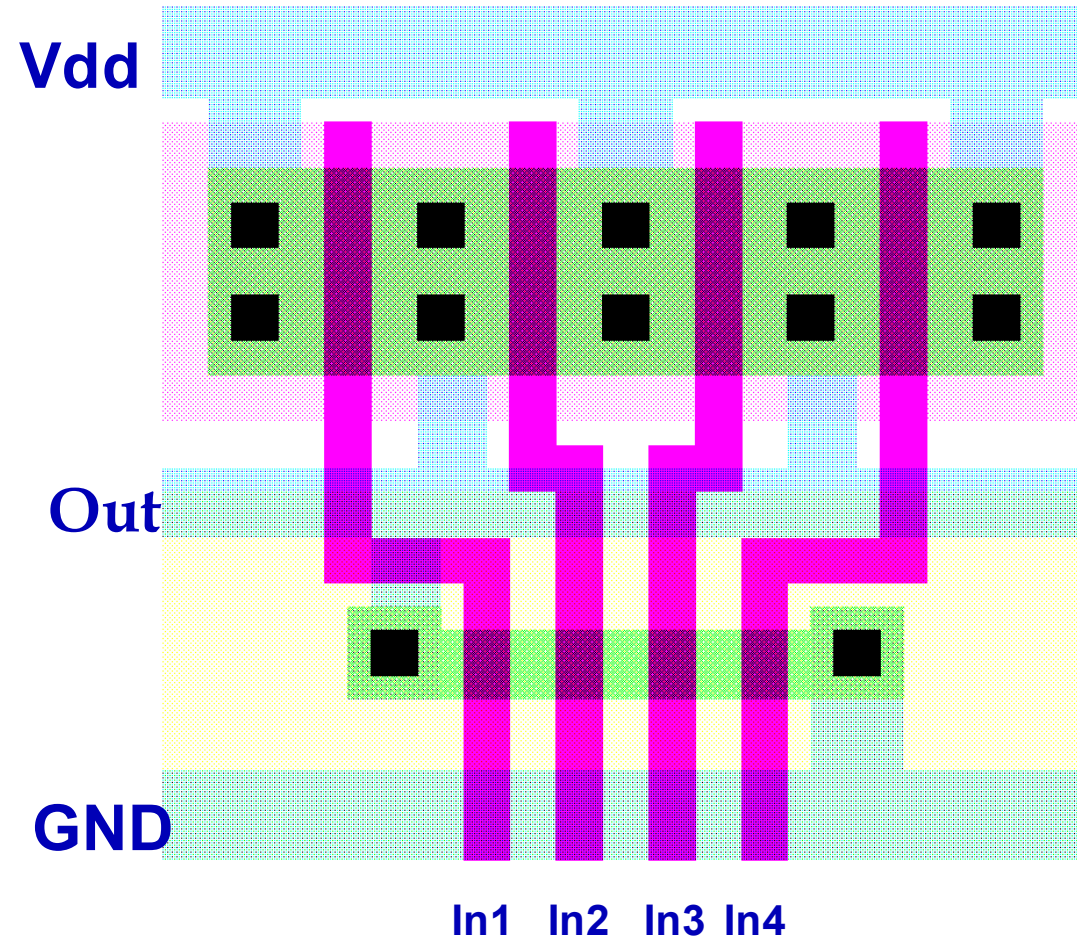
$$f = a.s + b.\overline{s}$$

$$f = \overline{\overline{a.s + b.\overline{s}}} \longrightarrow f = \overline{(\overline{a + s}).(\overline{b + s})}$$



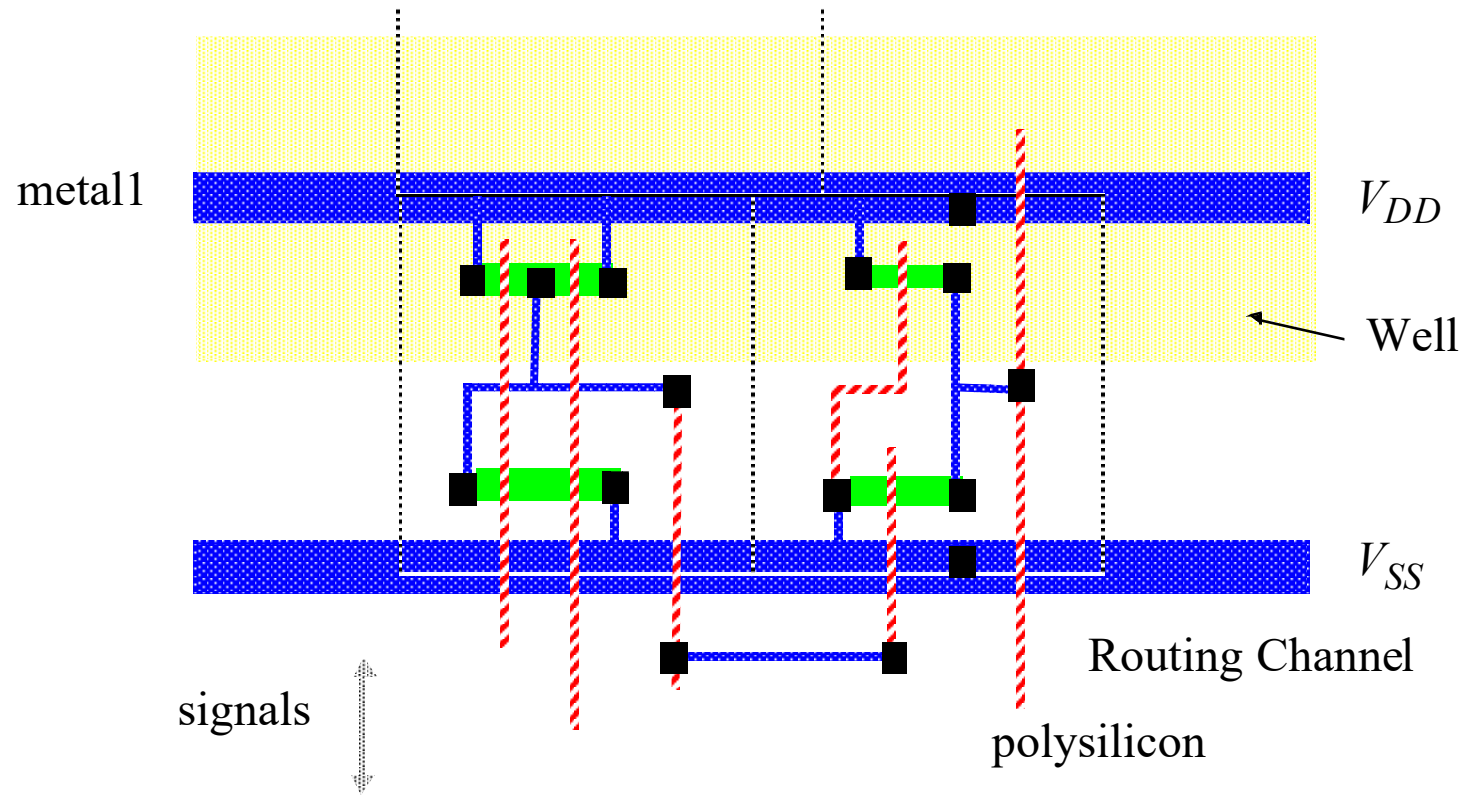
Examples of layouts of CMOS

Complex gates

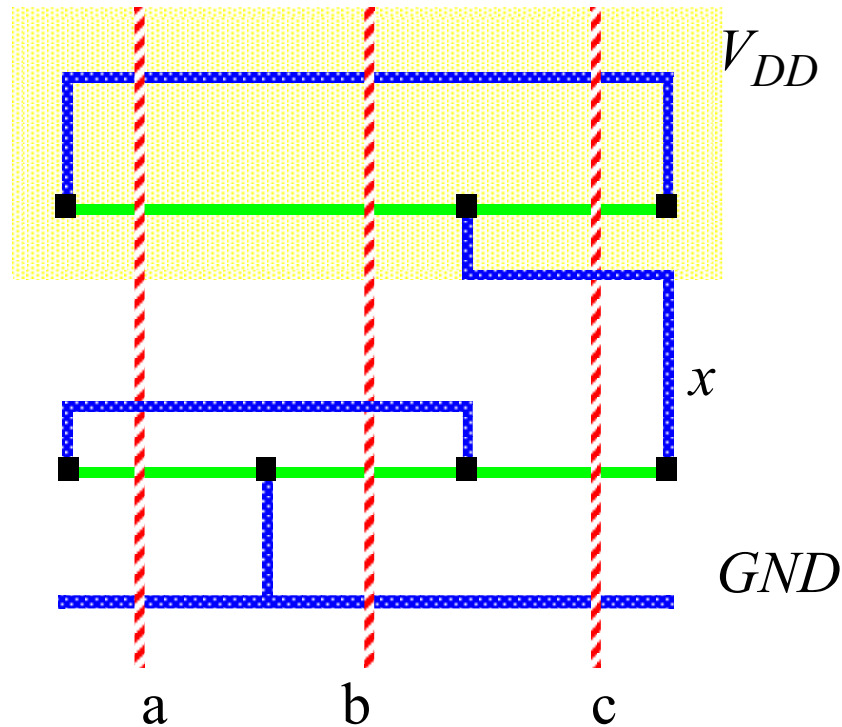


4-input NAND Gate

Examples of Standard Cell Layout Methodology



Examples of Standard Cell Layout



$$f = \overline{(a+b)} \cdot c$$

The Good & Bad News

❖ High Noise margins:

❖ V_{OH} and V_{OL} are at V_{DD} and GND

❖ No Static Power dissipation

❖ There is no direct path between V_{DD} and GND in steady state mode

❖ Comparable rise and fall times

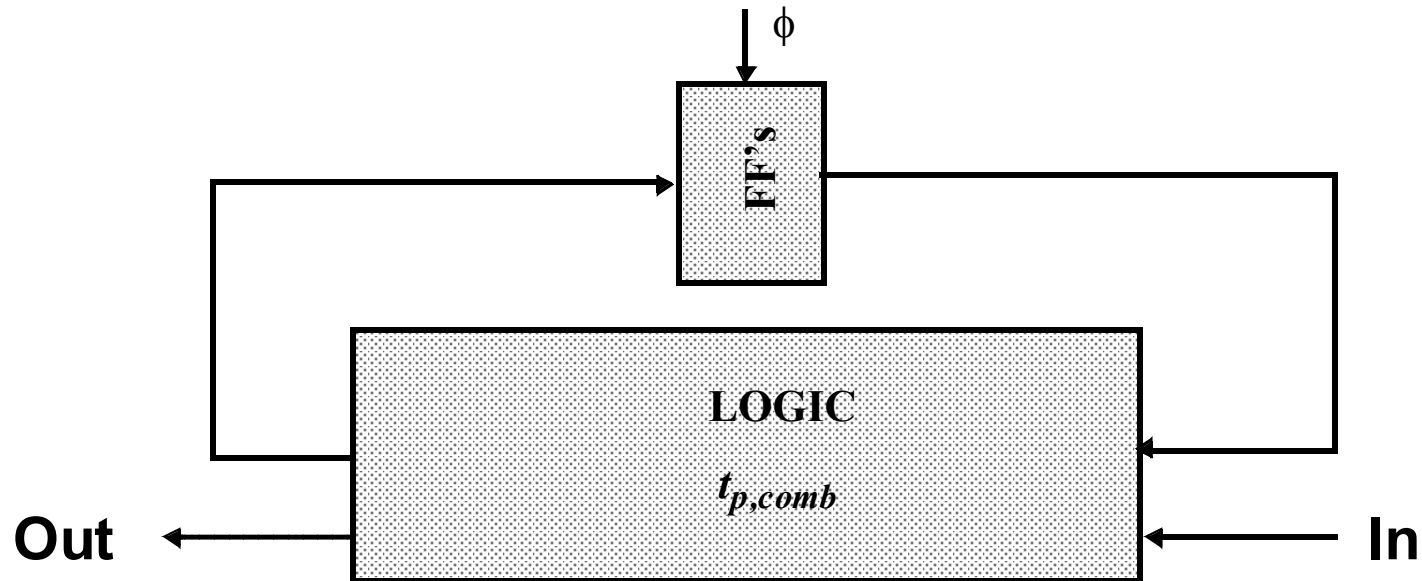
❖ (under appropriate scaling conditions)

- Slows down dramatically for large fan-ins due to long series stack of transistors
 - Fan-ins = number of inputs
 - PMOS stacks are worse than NMOS series stacks
- Large number of transistors
 - $2n$ devices for n -input NAND
 - At least 2 devices per input
- Bigger layout
 - N+ and P+ spacing rule and well spacing rule
 - Large device sizes required to counteract series stack
- Limit fan-in to 3 or 4...or delay and area will be too large

Combinational vs. Sequential Logic

- Combinational logic
 - Output is directly related to the input
 - No feedback loop (no memory!!)
- Sequential Logic Circuits
 - New output depends on the inputs and preceding values of output (has memory)
 - Static sequential circuits have one or more outputs connected back to input in positive feedback (regeneration)

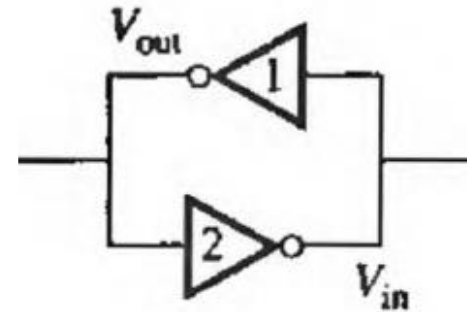
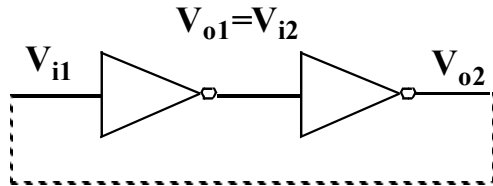
Sequential Logic



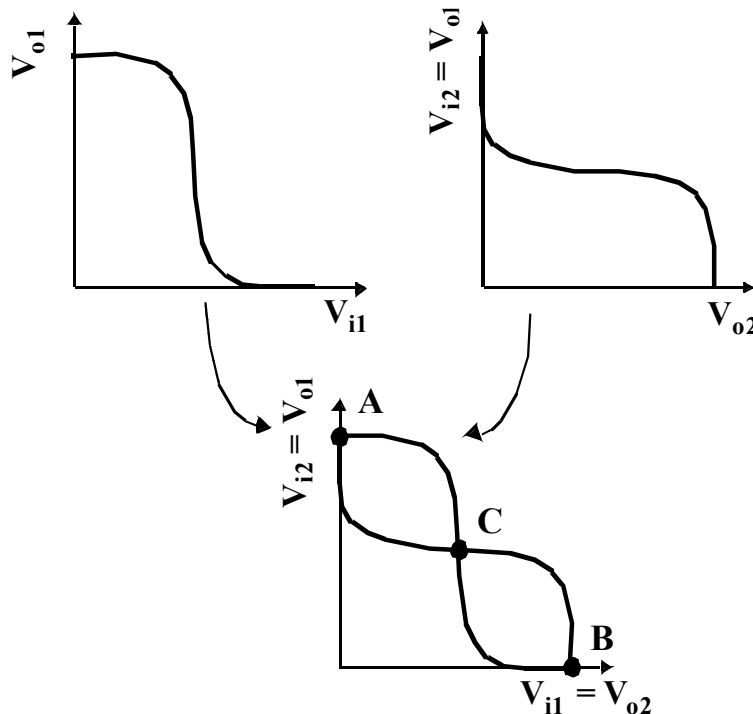
2 storage mechanisms

- **positive feedback**
- **charge-based**

Positive Feedback: Bi-Stability

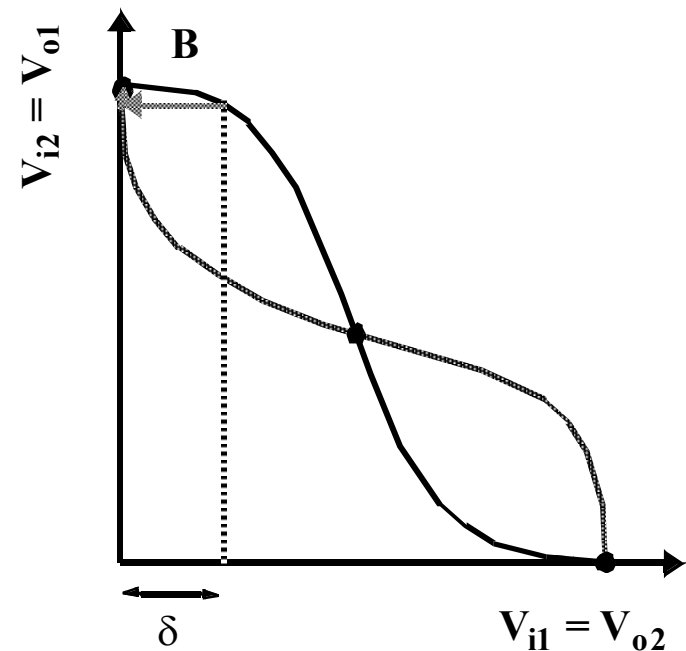
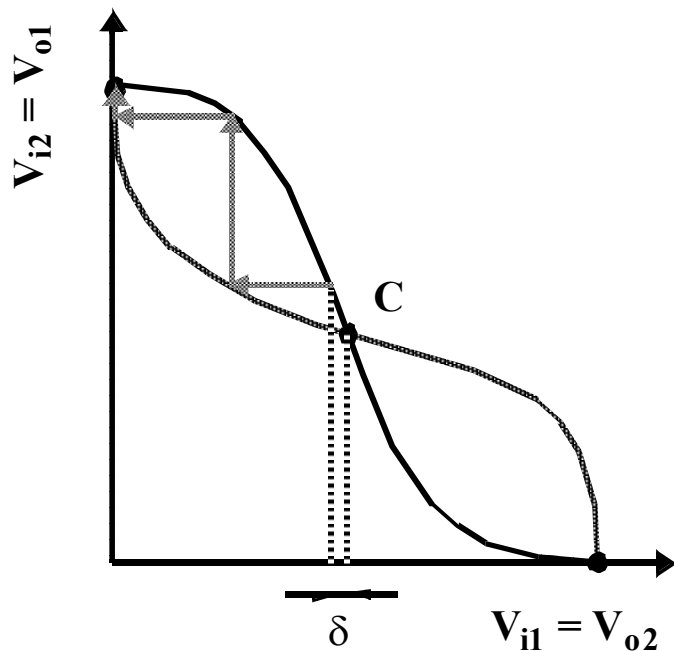


Cross-coupled inverters



A, B $\rightarrow$ stable operating point
C $\rightarrow$ unstable

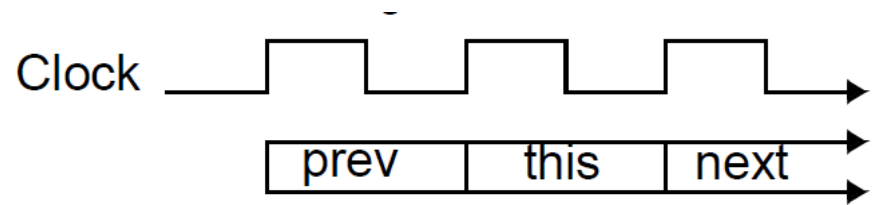
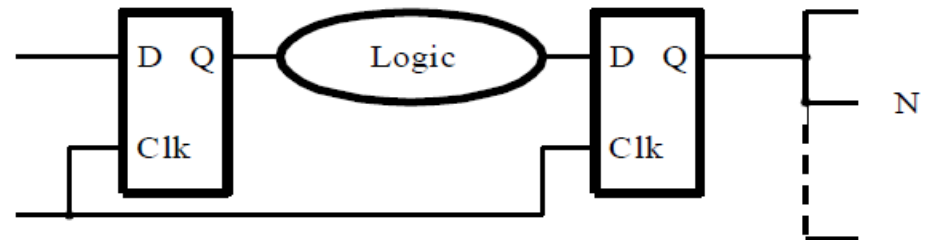
Meta-Stability



Gain should be larger than 1 in the transition region

Examples of Bi-Stable Circuits

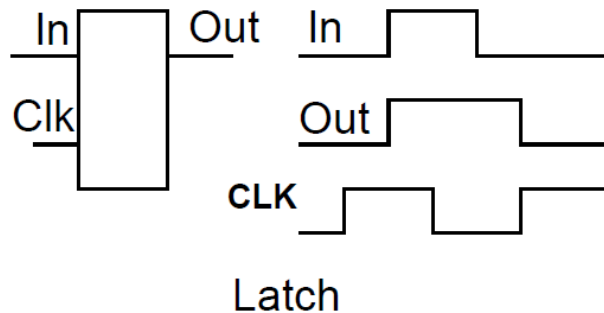
- Flip-flops and latches are logic elements used for storage
- We typically build finite state machines from combinational logic (next state logic) and latches or flip-flops (storage elements) to store the state information.



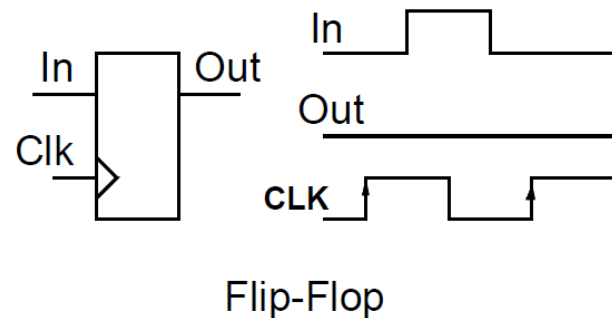
- We then control latches and flip-flops with a clock to create synchronous logic circuits. The clock ensures that we can tell the difference between previous, current and future states of the logic circuit

Latches vs. Flipflops

- **Latch (level-sensitive, transparent)** When the clock is high it passes **In** value to **Out**
- When the clock is low, it holds value that **In** had when the clock fell



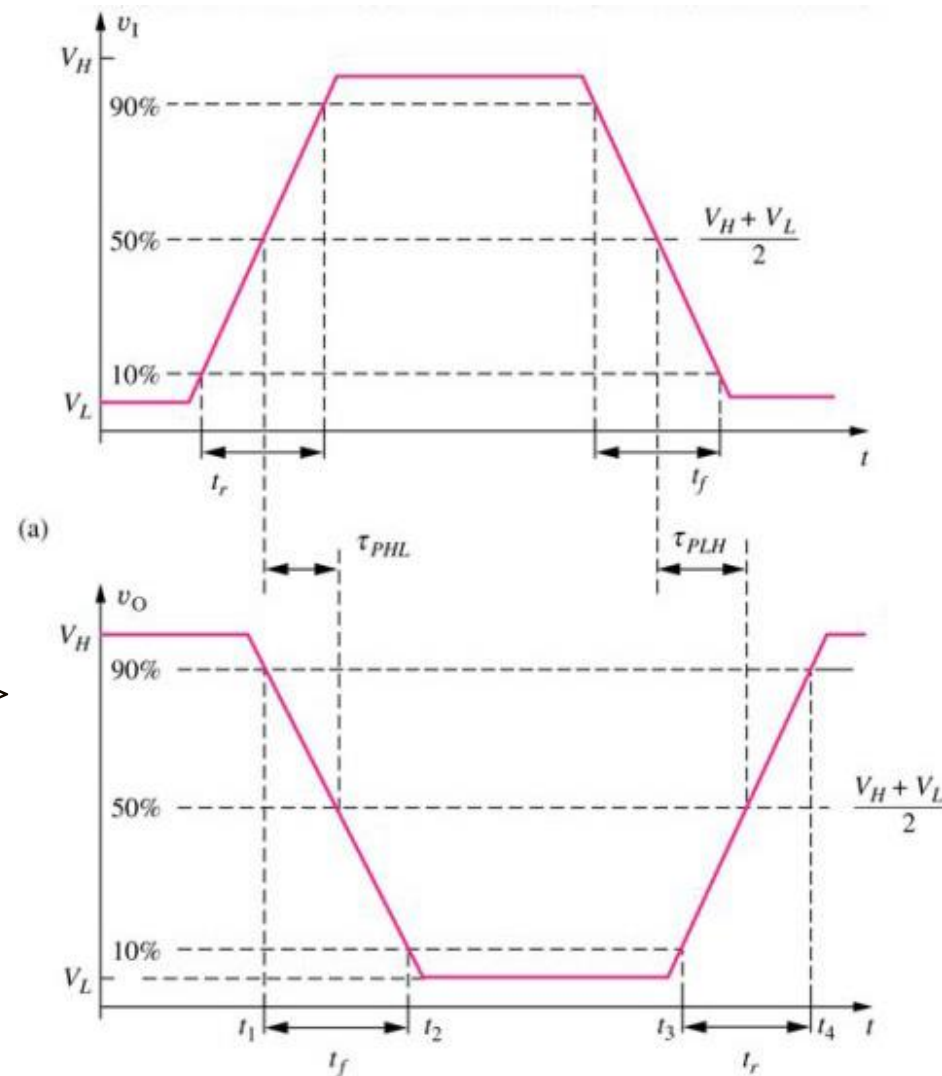
- **Flip-Flop (edge-triggered, non transparent).**
- On the *rising* edge of clock (pos-edge trig), it transfers the value of **In** to **Out**
- It holds the value at all other times.



Switching States in a Bi-Stable Ckt.

1. For the bi-stable ckt to change state $\rightarrow$ gain > 1
2. Introduce a trigger pulse @ V_{in}
3. Amplitude of trigger pulse sufficient to raise the voltage beyond V_S (point @ which gain > 1)
4. **Width of the trigger pulse** $>$ total propagation delay around the circuit loop ($2\tau_p$)

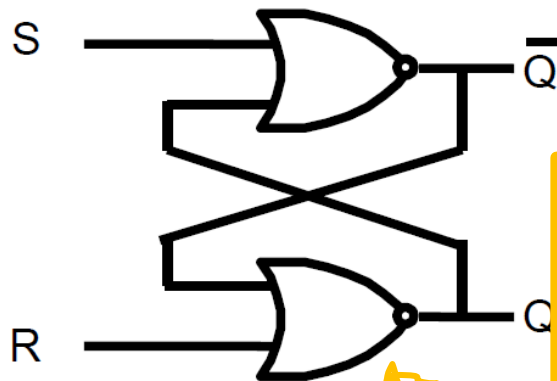
$$\tau_p = \frac{\tau_{PHL} + \tau_{PLH}}{2}$$



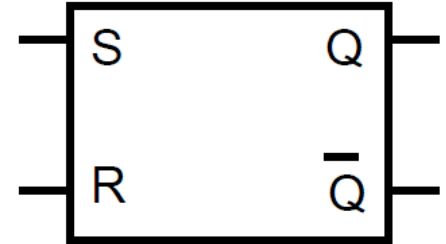
SR Latch with NOR-Gates

Simplest FF is a **cross-coupled pair of NOR gates**

- When $S=1$, $Q=1$
- When $R=1$, $Q=0$
- **By setting both $S=0$ and $R=0$, the previous state is held**
- Illegal state occurs when $R=1$ and $S=1$ (actually, the final state is determined by which signal goes low last)



0	0	1
0	1	0
1	0	0
1	1	0

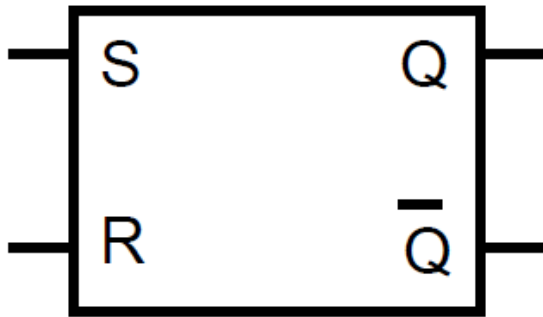


S	R	Q	$\bar{Q}$
0	0	Q	$\bar{Q}$
0	1	0	1
1	0	1	0
1	1	0	0

SR Latch with NOR-Gates

Simplest FF is a **cross-coupled pair of NOR gates**

- When $S=1, Q=1$
- When $R=1, Q=0$
- **By setting both $S=0$ and $R=0$, the previous state is held**
- Illegal state occurs when $R=1$ and $S=1$ (actually, the final state is determined by which signal goes low last)



S	R	Q	$\bar{Q}$
0	0	Q	$\bar{Q}$
0	1	0	1
1	0	1	0
1	1	0	0

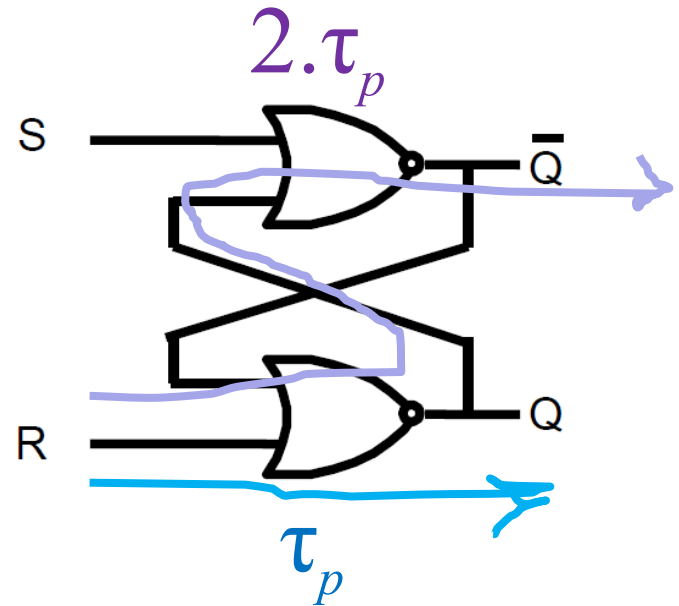
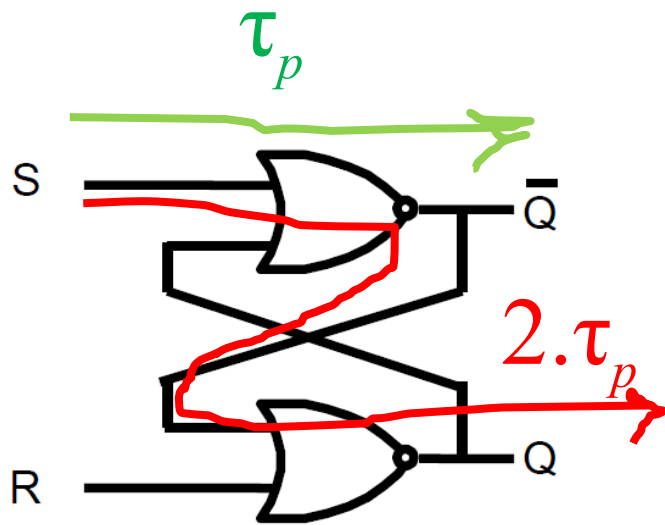
→ HOLD state

→ SET

→ RESET

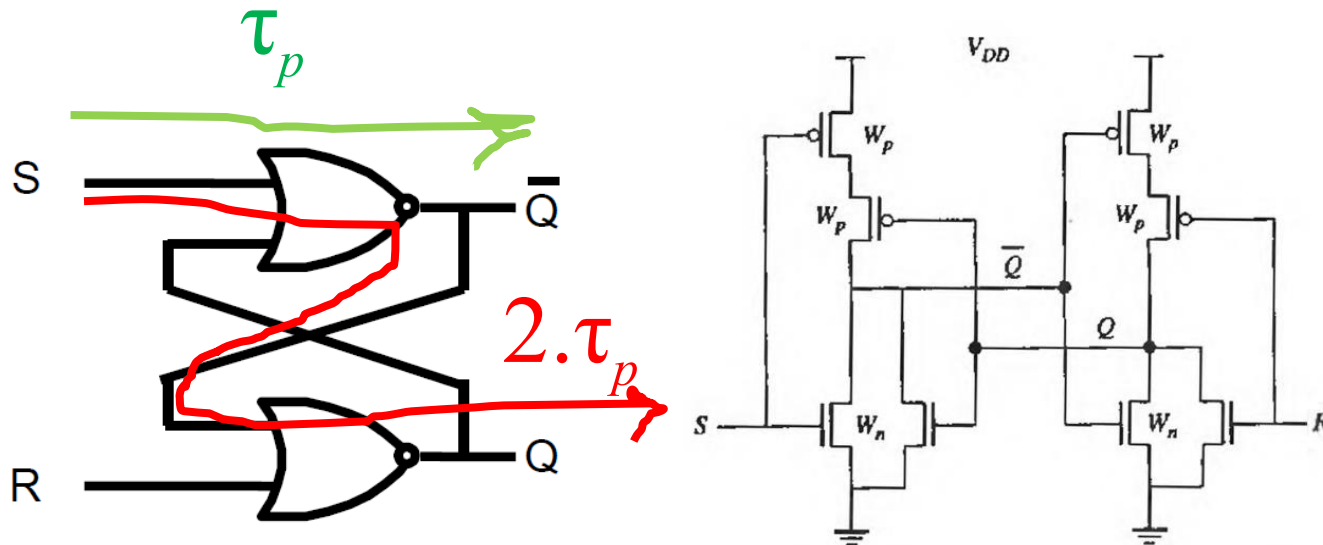
→ Illegal

Delays in SR Latch with NOR-Gates

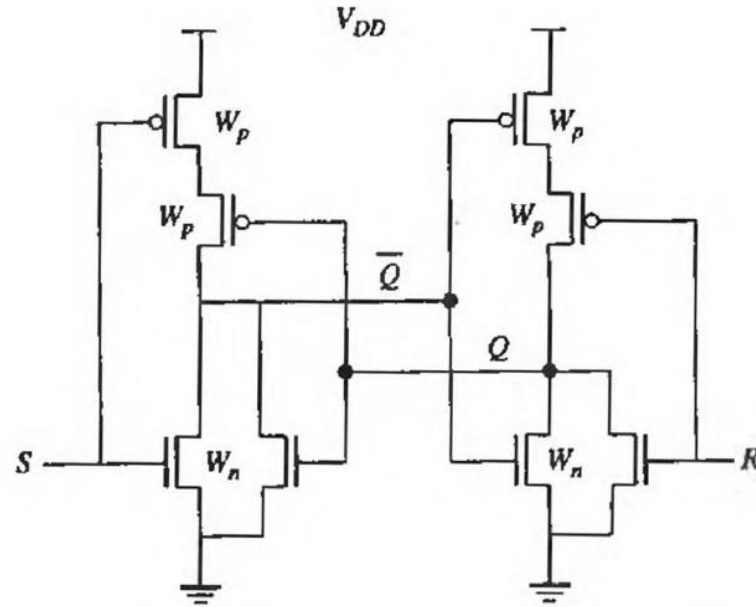
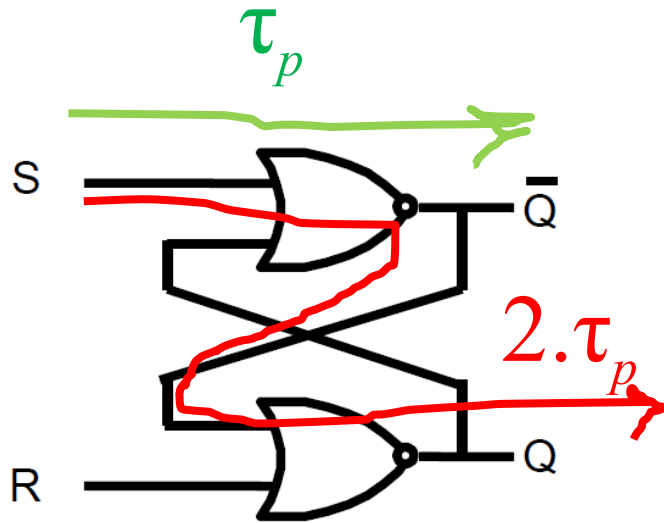


Design Example (S-R Latch)

Design an SR latch in 0.13 μm CMOS using NOR gates to deliver a delay of 400ps from S to Q and from R to $\bar{Q}$. Assume that the total load to be driven by Q and $\bar{Q}$ is 100 fF, and $L = 100$ nm.



Design Example (S-R Latch)



$$\tau_1 = \tau_2 = 200 \text{ ps}$$

$$t_{PHL} = t_{PLH}$$

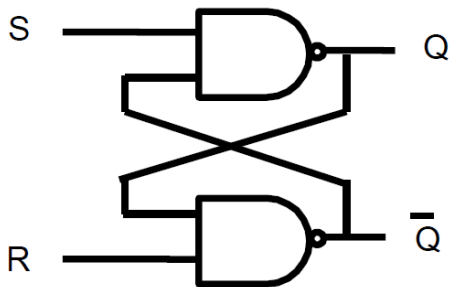
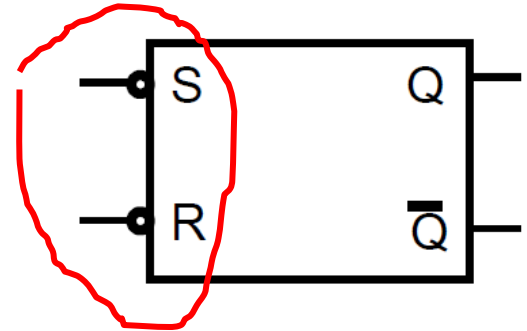
$$t_{PHL} = 0.7 * R_{eff} * C_L$$

$$R_{eff} = 12.5 * \left(\frac{L}{W}\right)_n$$

SR Latch with NAND-Gates

Simplest FF is a **cross-coupled pair of NOR gates**

- When $S=0$, $Q=1$
- When $R=0$, $Q=0$
- By setting both $S=1$ and $R=1$, the previous state is held
- Illegal state occurs when $R=0$ and $S=0$ (actually, the final state is determined by which signal goes low last)

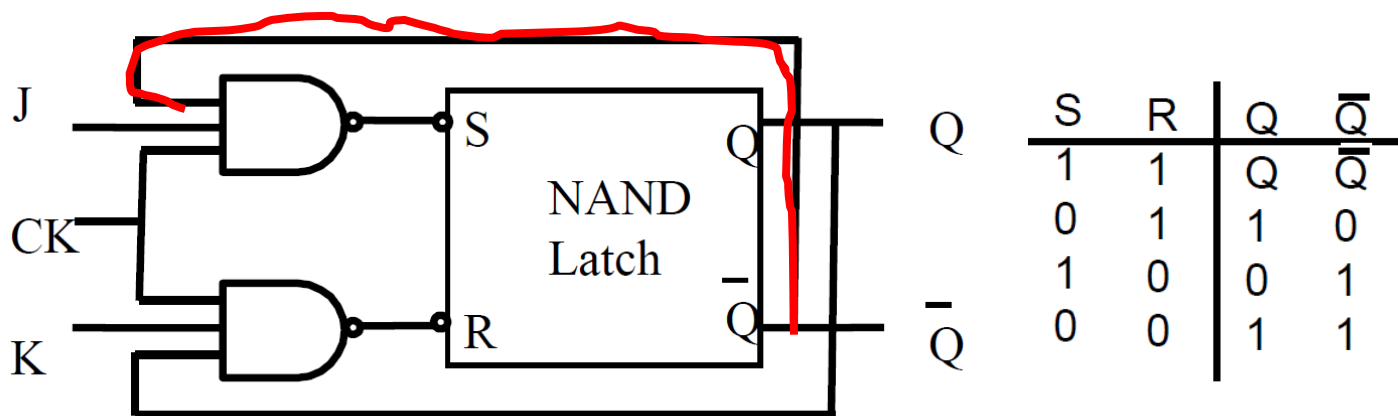


0	0	1
0	1	1
1	0	1
1	1	0

S	R	Q	$\bar{Q}$	
1	1	Q	$\bar{Q}$	→ HOLD state
0	1	1	0	→ SET
1	0	0	1	→ RESET
0	0	1	1	→ Illegal

JK Flip-Flop

- To avoid illegal state, use JK flip-flop
- In NAND implementation, $J=K=1$ flips the state of the output
- Clock is used to enable the output
- Will oscillate if clock is high too long when $J=K=1$

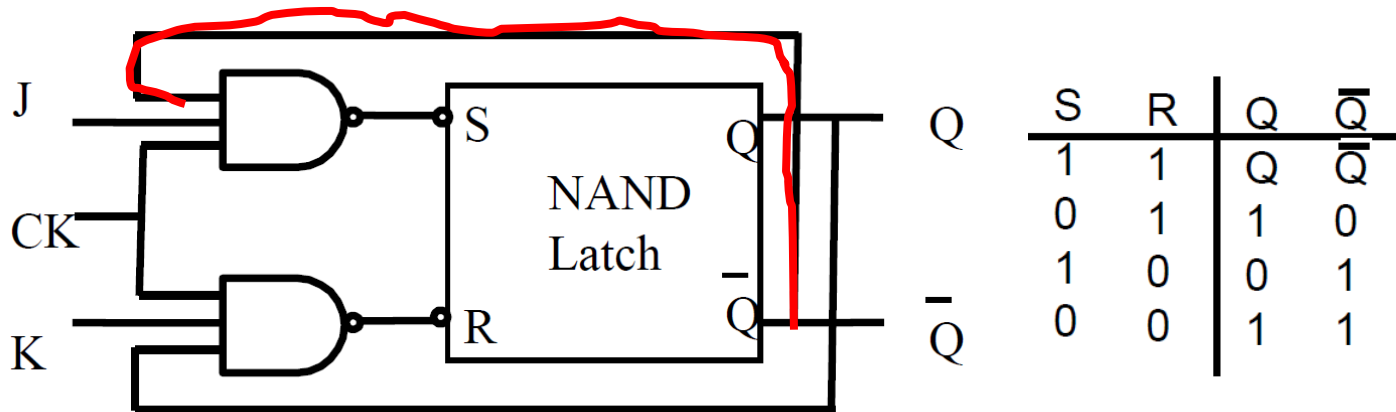


J _n	K _n	Q _{n+1}
0	0	Q _n
0	1	0
1	0	1
1	1	$\bar{Q}_n$

Two feedback lines remove the ambiguity when $R=S=0$

JK Flip-Flop

- J – k $\rightarrow$ synchronous inputs
- J $\rightarrow$ clocked-set input
- K $\rightarrow$ clocked –reset input



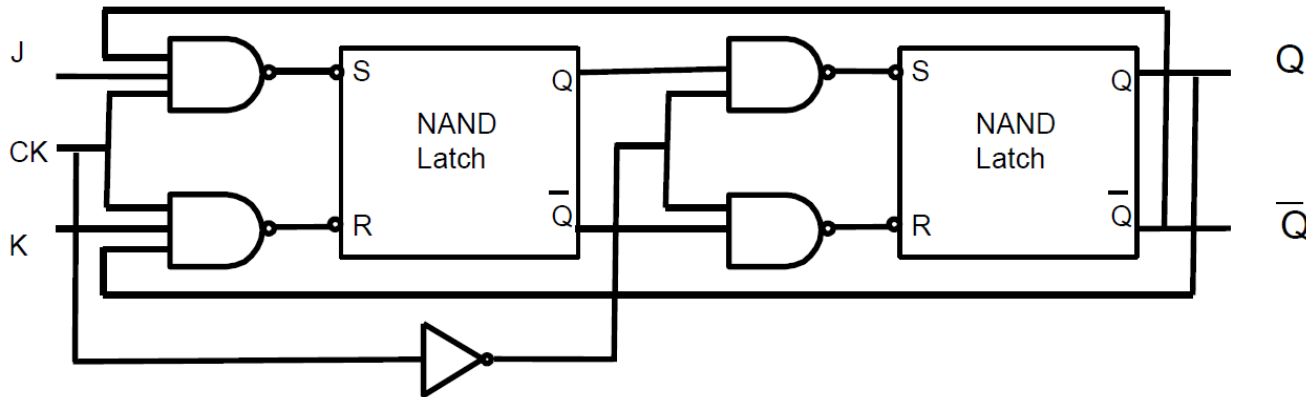
S	R	Q	$\bar{Q}$
1	1	Q	$\bar{Q}$
0	1	1	0
1	0	0	1
0	0	1	1

J_n	K_n	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\bar{Q}_n$

Two feedback lines remove the ambiguity when $R=S=0$

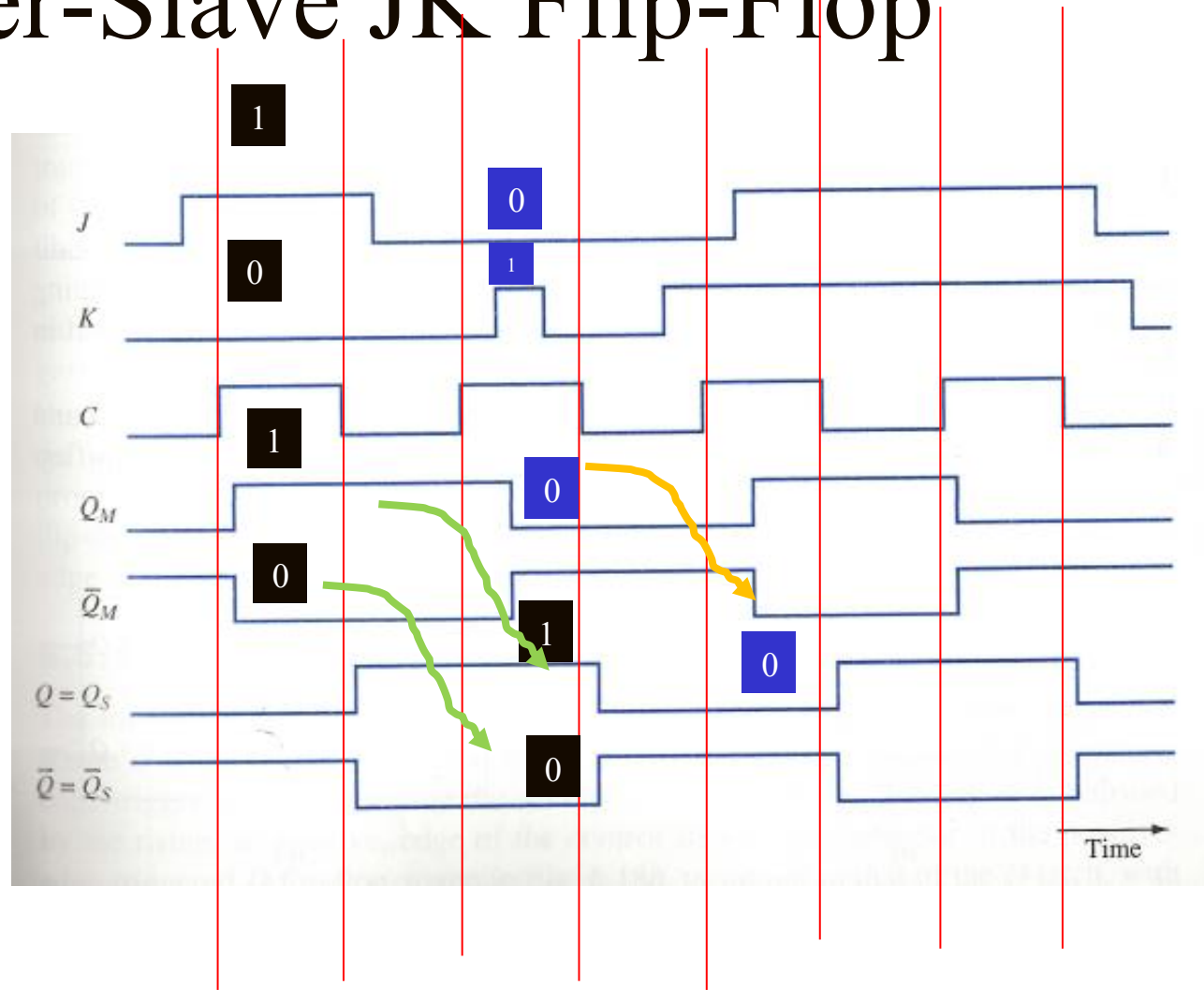
JK Master-Slave Flip-Flop

- Cascade of two JK Flip-flops
- Master activated by CK, Slave activated by $\overline{\text{CK}}$
- Master latches new data, slave launches old data
- As CK rises, $\overline{\text{CK}}$ goes down $\rightarrow$ CK has fallen sufficiently to disable the input NAND gates of the slave. This isolates the slave from the master and freezes the state of the slave latch.
- Due to this master-slave principle, there is no limit to how wide CK can be since, when the output changes state, entry into the master has been disabled.
- There is a minimum limit to the width of CK. It must be wider than the propagation delay time through the master flip-flop for proper functionality.



Master-Slave JK Flip-Flop

Inputs			Outputs	
J	K	C	Q^+	$\bar{Q}^+$
0	0		Q	$\bar{Q}$
0	1		0	1
1	0		1	0
1	1		$\bar{Q}$	Q
X	X	0	Q	$\bar{Q}$



Timing Diagram for Master-Slave JK Flip-Flop

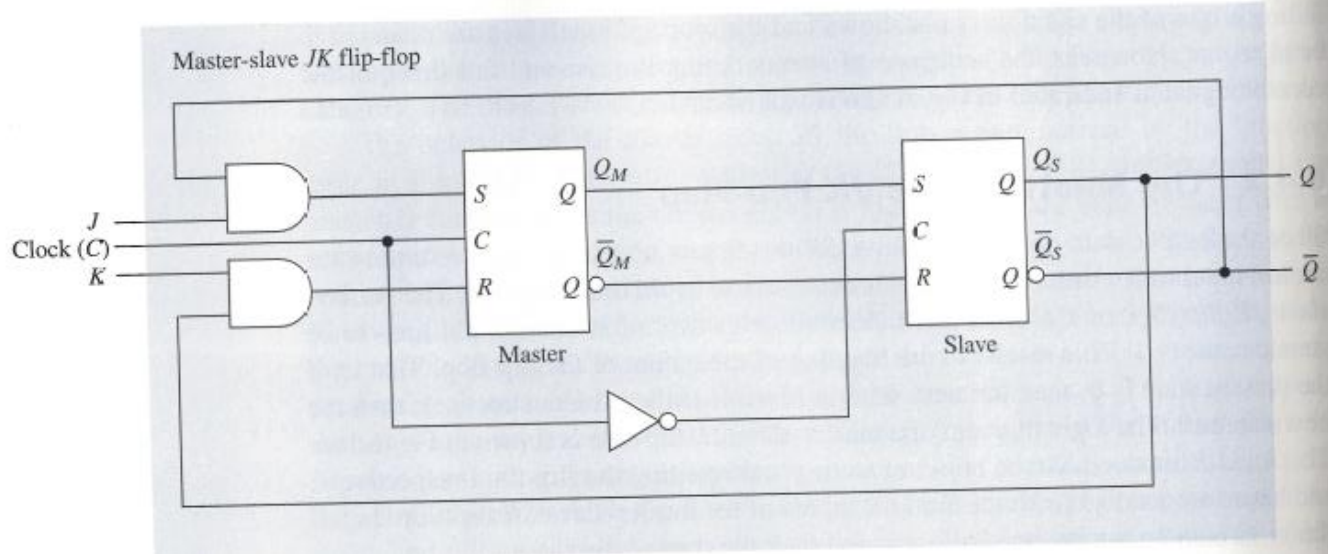
An RS latch is made of two NOR gates that hold state when the inputs are 0. If Set transitions to 1 then it gets set. If it subsequently transitions back to 0 it is still set at 1. If the Set glitches and jumps to 1 and then back to 0 quickly the 1 will still be held by the latch. It catches the one even though it may be a quick transition.

A flipflop may catch zeros if it is made of NAND gates where 1's will make the latch hold state and a zero will set the latch. If there is a glitch, the latch may catch a zero, therefore it is possible for a Master-Slave flip-flop to do “zeros catching.”

0's and 1's Catching

- The master is enabled during the entire period the control-signal is 1.
- If the slave latch is in its 1-state, then a logic-1 on K-input line causes the master-latch to reset. Slave becomes reset when control signal returns to 0.
- This is known as 0's catching (2nd pulse).
 - Note: if a subsequent 1-signal on J input line and C is still 1, master does not become set again (due to feedback not changing).
- If slave latch is in 0-state, logic-1 on J input line while control signal is 1 causes the master latch to be set and slave will be set upon occurrence of the falling edge.
- This is known as 1's catching (3rd pulse).
- In many applications, 0's and 1's catching behavior is undesirable. Normally recommended that the J and K input values should be held fixed during the entire interval the master is enabled.
- Any changes in J, K must occur while the control signal is 0.

0's Catching



Inputs			Outputs	
J	K	C	Q^+	$\bar{Q}^+$
0	0		Q	$\bar{Q}$
0	1		0	1
1	0		1	0
1	1		$\bar{Q}$	Q
X	X	0	Q	$\bar{Q}$

- Assume in 1-state ($Q = 1, \bar{Q} = 0$), $C = 1, J = 0, K = 0$
- K gets set to 1 briefly.
 - Master gets reset, Slave will become reset when Clock goes to 0.
- K goes to 0.
- J goes to 1. What happens?
- Nothing! Slave will still become reset when Clock goes to 0.

Response to illustrate 0's catching:

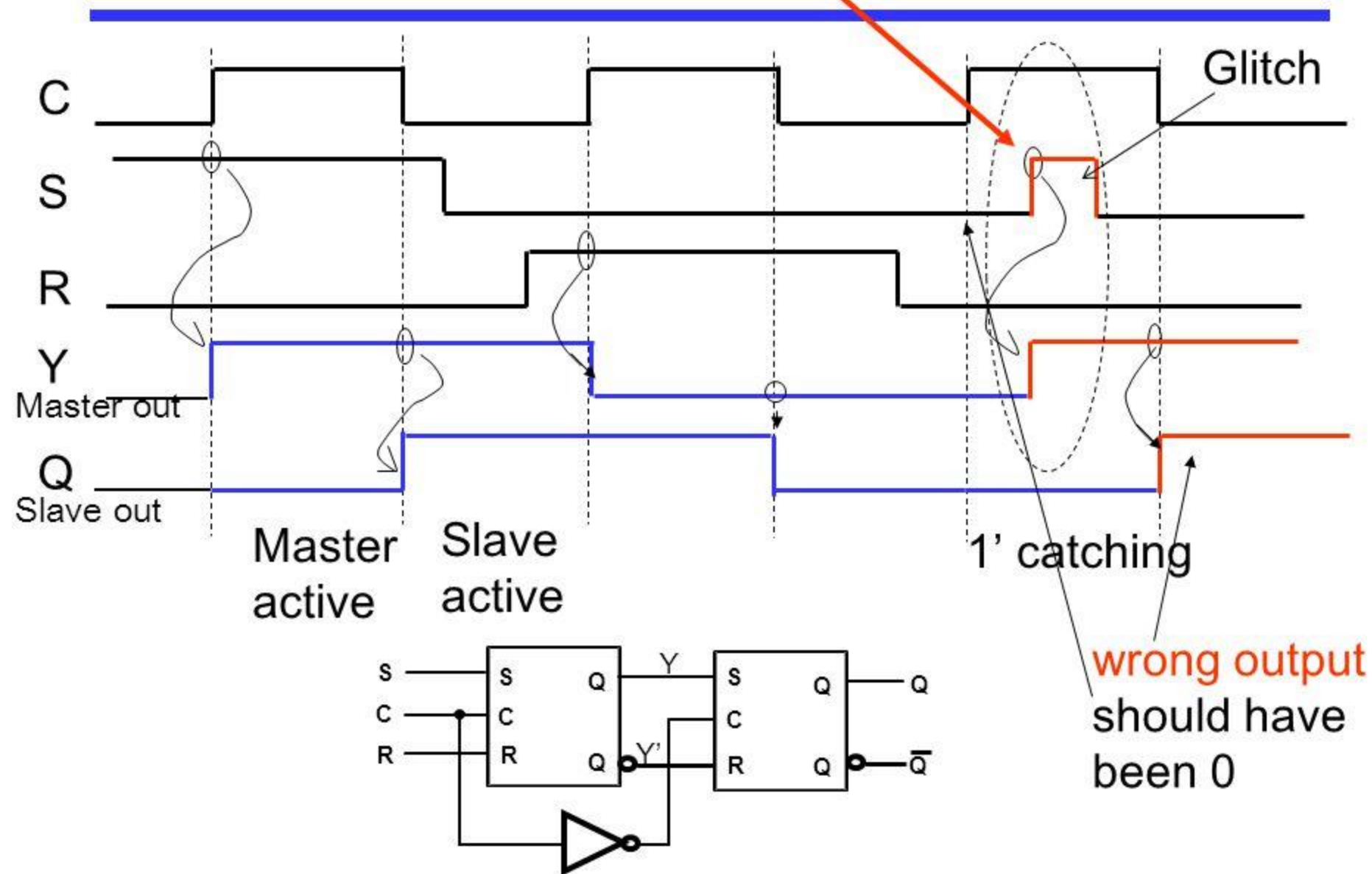
Initially, let flip-flop be SET i.e. $Q = 1$, $\bar{Q} = 0$

0's and 1's Catching Problem

1,857 views · Nov 15, 2020

Mellow outlook

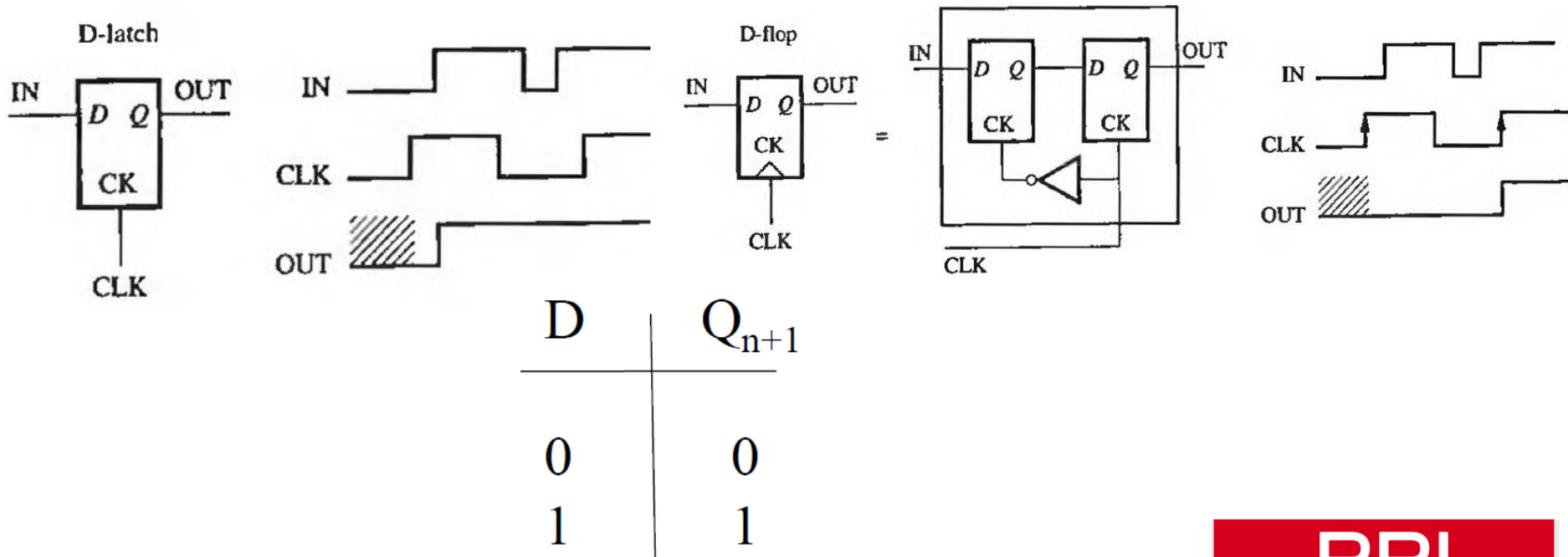
Flip-Flop Problem: 1' catching



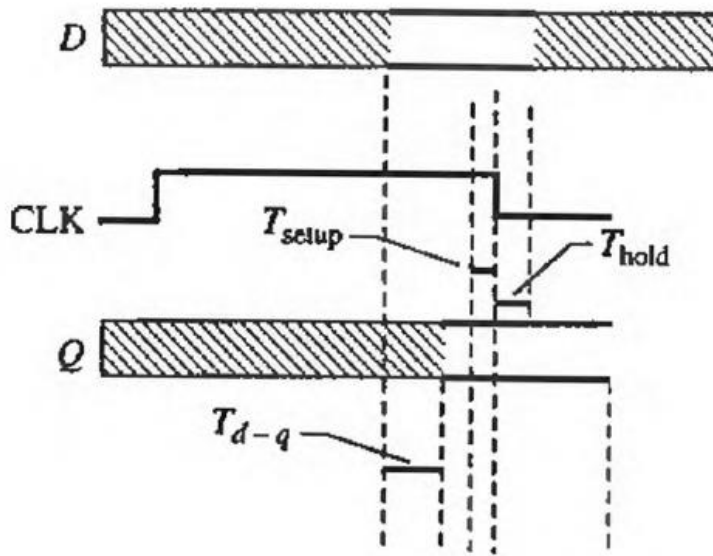
D-Flip Flop

Very useful FF

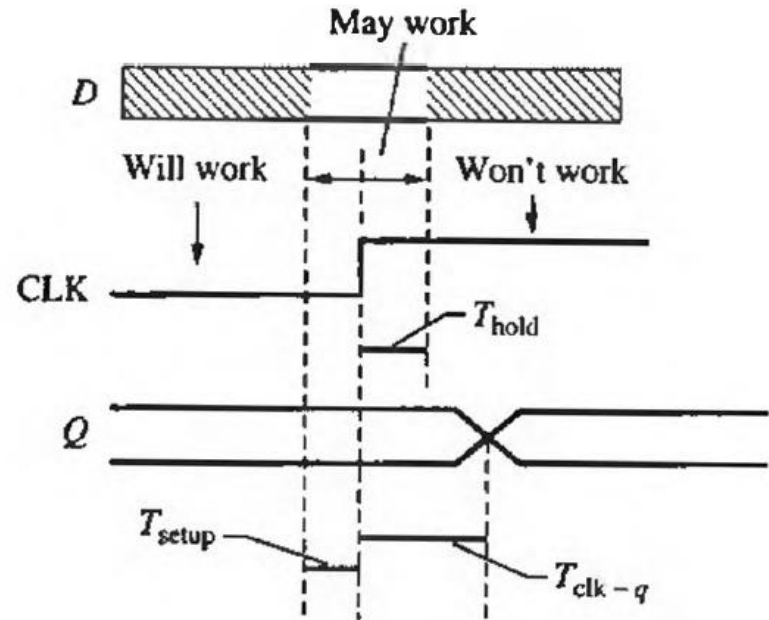
- Widely used in IC design for temporary storage of data
- May be *edge-triggered* (Flip-flop) or *level-sensitive* (*transparent D-latch*)



Timing requirements (Latch vs FF)



Latch



Flip Flop

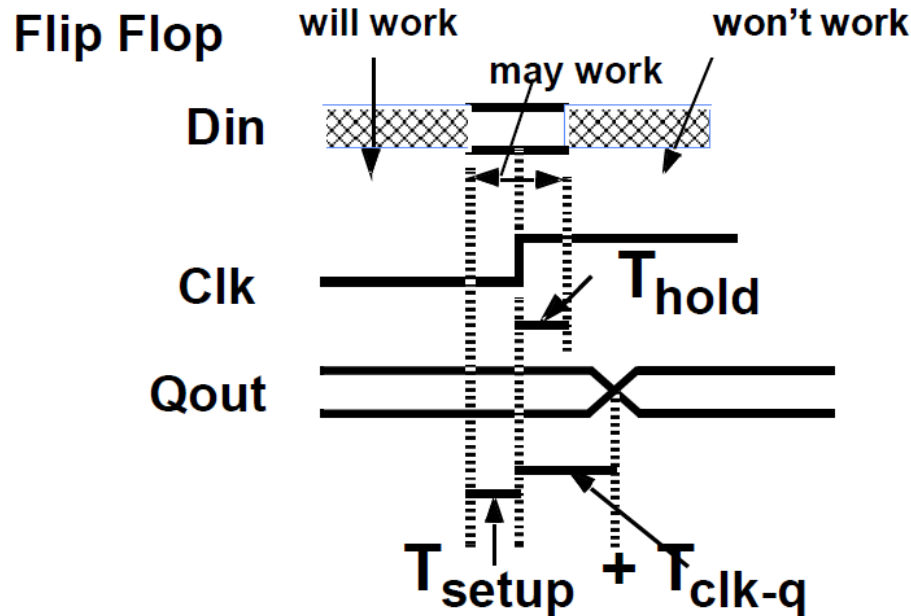
T_{setup} : time that the incoming data must be stable before the clock arrives.

T_{hold} : length of time that the data remains stable after the clock arrives for proper operation

$T_{\text{CLK-Q}}$: delay from the time that the clock arrives to the point at which the Q output stabilizes.

Flip Flop Clocking Overhead

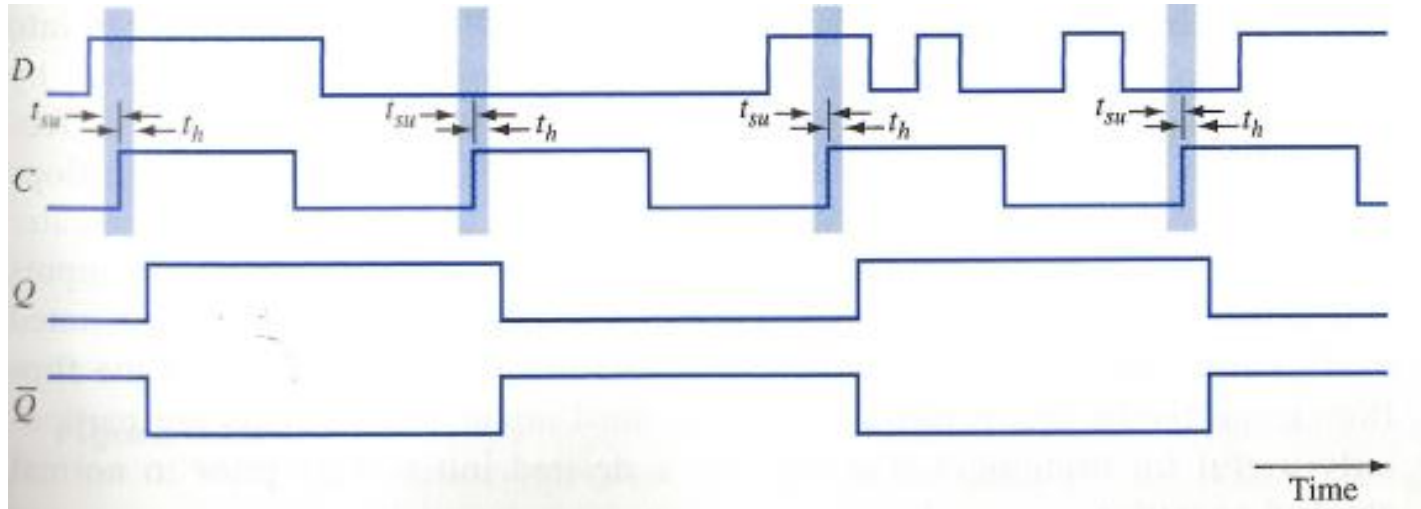
FF have setup and hold times that must be satisfied:



If Din arrives before setup time and is stable after the hold time, FF will work; if Din arrives after hold time, it will fail; in between, it may or may not work; FF delays the slowest signal by the setup + clk-q delay in the worst case

$$\text{Overhead} = T_{setup} + T_{CLK_q}$$

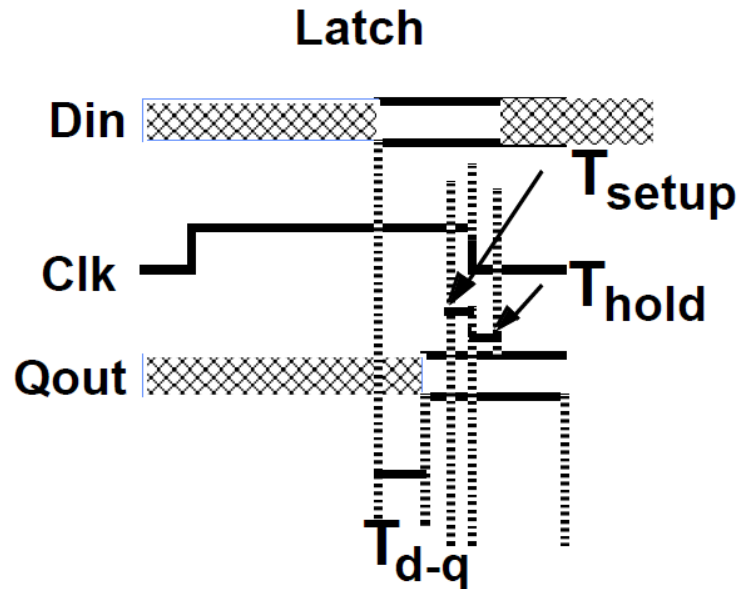
Timing Diagram



During setup and hold times t_{setup} , t_{hold} with respect to the triggering edge of the clock, D input must not change.

Latch Clocking Overhead

Latches also have setup and hold times that must be satisfied:



But latch has small setup and hold times; however, it delays the late arriving signals by $T_{\text{d-q}}$ and this is more important than the setup and hold times.

Gate and Transistor Level Realization

